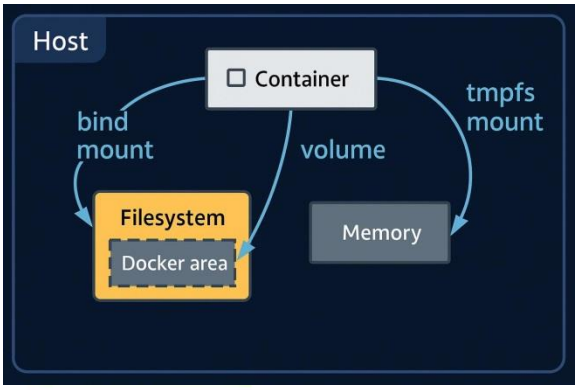
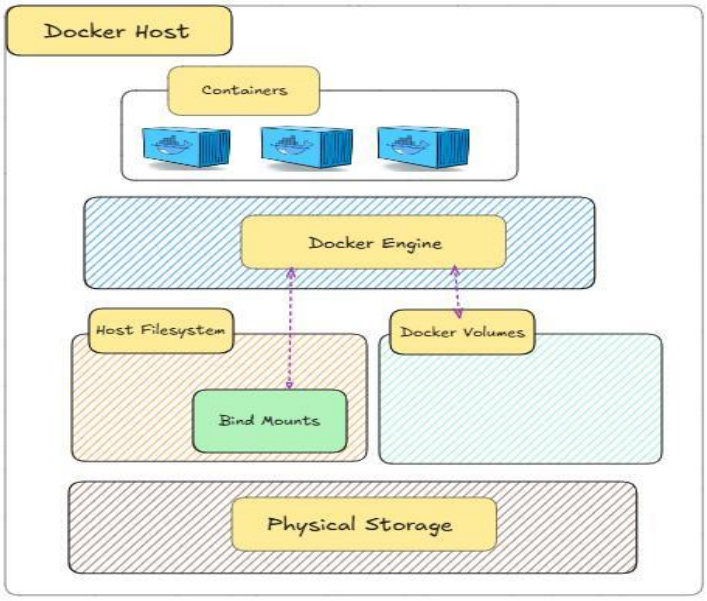
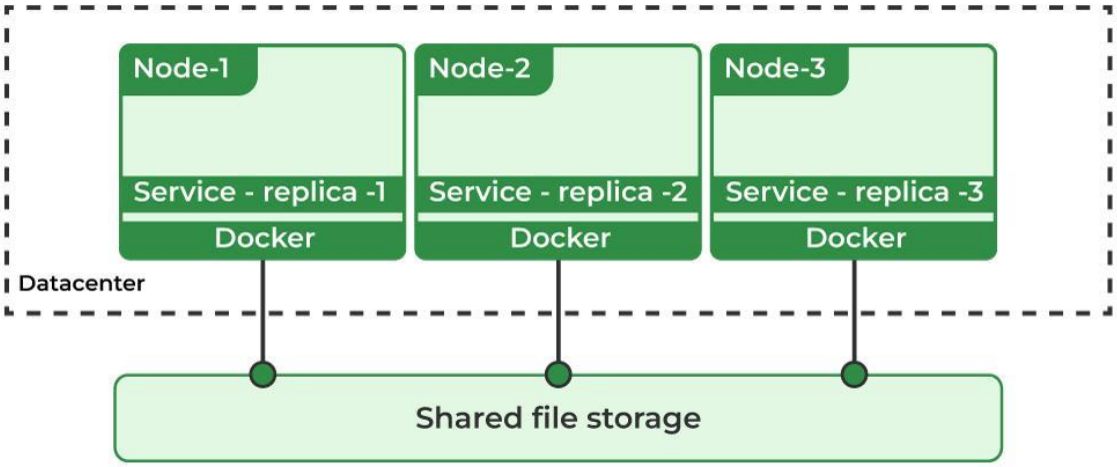
Docker Volumes:







1. What is a Docker Volume?

A **Docker Volume** is a mechanism for **persistent data storage** used by containers.

Normally:

- Container data is stored inside the container filesystem.
- When the container is **deleted**, the data is **lost**.

Docker volumes solve this problem by storing data **outside the container** on the host system.

Definition

A Docker volume is a persistent storage location managed by Docker that allows containers to store and share data.

---

2. Why Docker Volumes Are Used

Containers are **temporary (ephemeral)**. When a container stops or is deleted, its data disappears.

Volumes provide:

Feature	Description
Persistence	Data survives container deletion
Data sharing	Multiple containers can share same data
Backup & migration	Easy to backup and restore
Performance	Faster than container writable layer
Isolation	Separate storage from container lifecycle

---

3. Docker Storage Types

Docker supports **three storage mechanisms**:

Storage Type	Description
Volumes	Managed by Docker (Recommended)
Bind Mounts	Direct host folder mounting
tmpfs	Temporary memory storage

Example comparison:

Feature	Volumes	Bind Mounts
Managed by Docker	Yes	No
Location	`/var/lib/docker/volumes`	Any host path
Portability	High	Low

---

4. Where Docker Volumes Are Stored

By default:

```
/var/lib/docker/volumes/
```

Example:

```
/var/lib/docker/volumes/myvolume/_data
```

---

5. Types of Docker Volumes

1 Named Volume

Volume created with a **name**.

Example:

```
docker volume create myvolume
```

Use in container:

```
docker run -d -v myvolume:/app/data nginx
```

Structure:

```
Volume Name : myvolume
Container Path : /app/data
```

---

2 Anonymous Volume

Volume created **automatically by Docker**.

Example:

```
docker run -d -v /app/data nginx
```

Docker creates random name like:

```
8fd92f6b7d3d...
```

Problem:

- Hard to manage.

---

3 Bind Mount

Mounts **host directory directly**.

Example:

```
docker run -v /home/ec2-user/data:/app/data nginx
```

Meaning:

```
Host folder → Container folder
```

---

6. Docker Volume Commands

List Volumes

```
docker volume ls
```

Output:

```
DRIVER    VOLUME NAME
local     myvolume
```

Create Volume

```
docker volume create myvolume
```

Inspect Volume

```
docker volume inspect myvolume
```

Output example:

```
[
 {
  "Name": "myvolume",
  "Driver": "local",
  "Mountpoint": "/var/lib/docker/volumes/myvolume/_data"
 }
]
```

Remove Volume

```
docker volume rm myvolume
```

Remove All Unused Volumes

```
docker volume prune
```

---

7. Using Volumes in Docker Run

Example 1 – Named Volume

```
docker run -d \
--name web \
-v myvolume:/usr/share/nginx/html \
nginx
```

Meaning:

```
myvolume → container html folder
```

Example 2 – Bind Mount

```
docker run -d \
-v /home/ec2-user/site:/usr/share/nginx/html \
nginx
```

---

8. Volume Sharing Between Containers

Multiple containers can share the same volume.

Example:

```
Container 1
Container 2
     ↓
   Volume
```

Command:

```
docker run -d --name container1 -v myvolume:/data nginx

docker run -d --name container2 -v myvolume:/data busybox
```

Both containers access **same data**.

---

9. Volumes in Dockerfile

Dockerfile example:

```
FROM nginx

VOLUME /usr/share/nginx/html
```

This creates an **anonymous volume**.

---

10. Volumes in Docker Compose

Example `docker-compose.yml`

```
version: '3'

services:

  web:
    image: nginx
    volumes:
      - myvolume:/usr/share/nginx/html

volumes:
  myvolume:
```

Run:

```
docker compose up -d
```

Docker Compose

1. What is Docker Compose

Docker Compose is a tool used to **define and run multi-container Docker applications**.

Instead of running many `docker run` commands manually, we define everything in **one YAML file** called:

```
docker-compose.yml
```

Then run all containers with one command.

Example use case:

```
Web Application
     |
     |---- Web Container (Node/Python)
     |
     |---- Database Container (MySQL/Postgres)
```

Compose starts both containers together.

---

2. Why Docker Compose

Without Compose:

```
docker network create mynet

docker run -d --name db \
-e MYSQL_ROOT_PASSWORD=root \
--network mynet mysql

docker run -d --name web \
-p 5000:5000 \
--network mynet myapp
```

With Compose:

```
docker compose up
```

Much simpler.

---

3. Docker Compose Architecture

Docker Compose uses **YAML configuration**.

Main components:

```
docker-compose.yml
        |
        |---- services
        |---- networks
        |---- volumes
```

Example structure

```
version: "3"

services:
  web:
  db:

volumes:

networks:
```

---

4. Installing Docker Compose

For Amazon Linux / CentOS

```
sudo yum install docker -y
sudo systemctl start docker
sudo systemctl enable docker
```

Install compose plugin:

```
sudo mkdir -p /usr/libexec/docker/cli-plugins

sudo curl -SL
https://github.com/docker/compose/releases/download/v2.27.0/docker-compose-
linux-x86_64 \
-o /usr/libexec/docker/cli-plugins/docker-compose

sudo chmod +x /usr/libexec/docker/cli-plugins/docker-compose
```

Check version:

```
docker compose version
```

---

5. Basic Docker Compose File

Example:

```
version: "3"

services:
  web:
    image: nginx
    ports:
      - "80:80"
```

Run:

```
docker compose up
```

Stop:

```
docker compose down
```

---

6. Important Sections in docker-compose.yml

1 version

Defines compose file version.

```
version: "3.8"
```

---

2 services

Defines containers.

Example:

```
services:
  web:
    image: nginx
```

---

3 image

Container image.

```
image: nginx
```

or build image:

```
build: .
```

---

4 ports

Port mapping.

```
ports:
 - "5000:5000"
```

Meaning:

```
HostPort : ContainerPort
```

---

5 volumes

Persistent storage.

```
volumes:
 - mydata:/data
```

---

6 environment

Environment variables.

```
environment:
  MYSQL_ROOT_PASSWORD: root
```

---

7 depends_on

Service dependency.

```
depends_on:
 - db
```

Web starts after DB.

---

8 networks

Custom network.

```
networks:
 - mynetwork
```

---

7. Example: Web + Database Application

```
version: "3"

services:

  web:
    image: nginx
    ports:
      - "8080:80"
    depends_on:
      - db

  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
```

Run:

```
docker compose up -d
```

Check containers:

```
docker ps
```

Stop:

```
docker compose down
```

---

8. Example: Python App + MySQL

Project structure:

```
project
   |
   |---- app.py
   |---- Dockerfile
   |---- docker-compose.yml
```

Dockerfile

```
FROM python:3.9

WORKDIR /app

COPY . .

RUN pip install flask mysql-connector-python

CMD ["python", "app.py"]
```

docker-compose.yml

```
version: "3"

services:

  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: root
```

Run:

```
docker compose up --build
```

---

9. Docker Compose Commands

Start containers

```
docker compose up
```

Detached mode

```
docker compose up -d
```

Stop containers

```
docker compose down
```

Build images

```
docker compose build
```

Restart services

```
docker compose restart
```

View running containers

```
docker compose ps
```

View logs

```
docker compose logs
```

Execute command inside container

```
docker compose exec web bash
```

Stop specific service

```
docker compose stop web
```

---

10. Docker Compose Networking

Compose automatically creates a network.

Example:

```
project_default
```

Containers communicate using **service names**.

Example:

```
web → db
```

Inside web container:

```
mysql -h db
```

---

11. Docker Compose Volumes

Example:

```
version: "3"

services:

  db:
    image: mysql
    volumes:
      - dbdata:/var/lib/mysql

volumes:
  dbdata:
```

This stores database data persistently.

---

12. Docker Compose Project Name

By default project name = **folder name**

Example:

```
project_web_1
project_db_1
```

Custom project name:

```
docker compose -p myproject up
```

---

13. Docker Compose File Version 3 Features

Supported features:

```
services
volumes
networks
configs
secrets
deploy
```

---

14. Docker Compose Workflow

Typical workflow:

```
1 Create application
2 Write Dockerfile
3 Create docker-compose.yml
4 Run docker compose up
5 Access application
6 Stop docker compose down
```

---

15. Real World Example

Example architecture:

```
Frontend (React)
Backend (Node)
Database (MongoDB)
Redis Cache
Nginx Reverse Proxy
```

Compose file runs **all containers together**.

---

16. Docker Compose vs Dockerfile

Feature	Dockerfile	Docker Compose
Purpose	Build image	Run multi containers
File	Dockerfile	docker-compose.yml
Used for	Image creation	Application orchestration
Command	docker build	docker compose up

---

17. Common Errors

Error

```
docker-compose: command not found
```

Fix:

Install compose plugin.

Port already used

```
Bind for 0.0.0.0:80 failed
```

Fix:

Change port.

Container name conflict

```
container name already in use
```

Fix:

```
docker compose down
```

---

18. Best Practices

✔ Use **.env files**
✔ Use **volumes for database**
✔ Use **depends_on**
✔ Use **custom networks**
✔ Use **multi-stage builds**

---

19. Docker Compose File Example (Professional)

```
version: "3.8"

services:

  web:
    build: .
```

```
    container_name: webapp
    ports:
      - "5000:5000"
    networks:
      - appnet
    depends_on:
      - db

  db:
    image: mysql:5.7
    container_name: database
    environment:
      MYSQL_ROOT_PASSWORD: root
    volumes:
      - dbdata:/var/lib/mysql
    networks:
      - appnet

volumes:
  dbdata:

networks:
  appnet:
```

---

20. Useful Commands Summary

```
docker compose up
docker compose up -d
docker compose down
docker compose ps
docker compose logs
docker compose build
docker compose restart
docker compose stop
docker compose start
docker compose exec
```